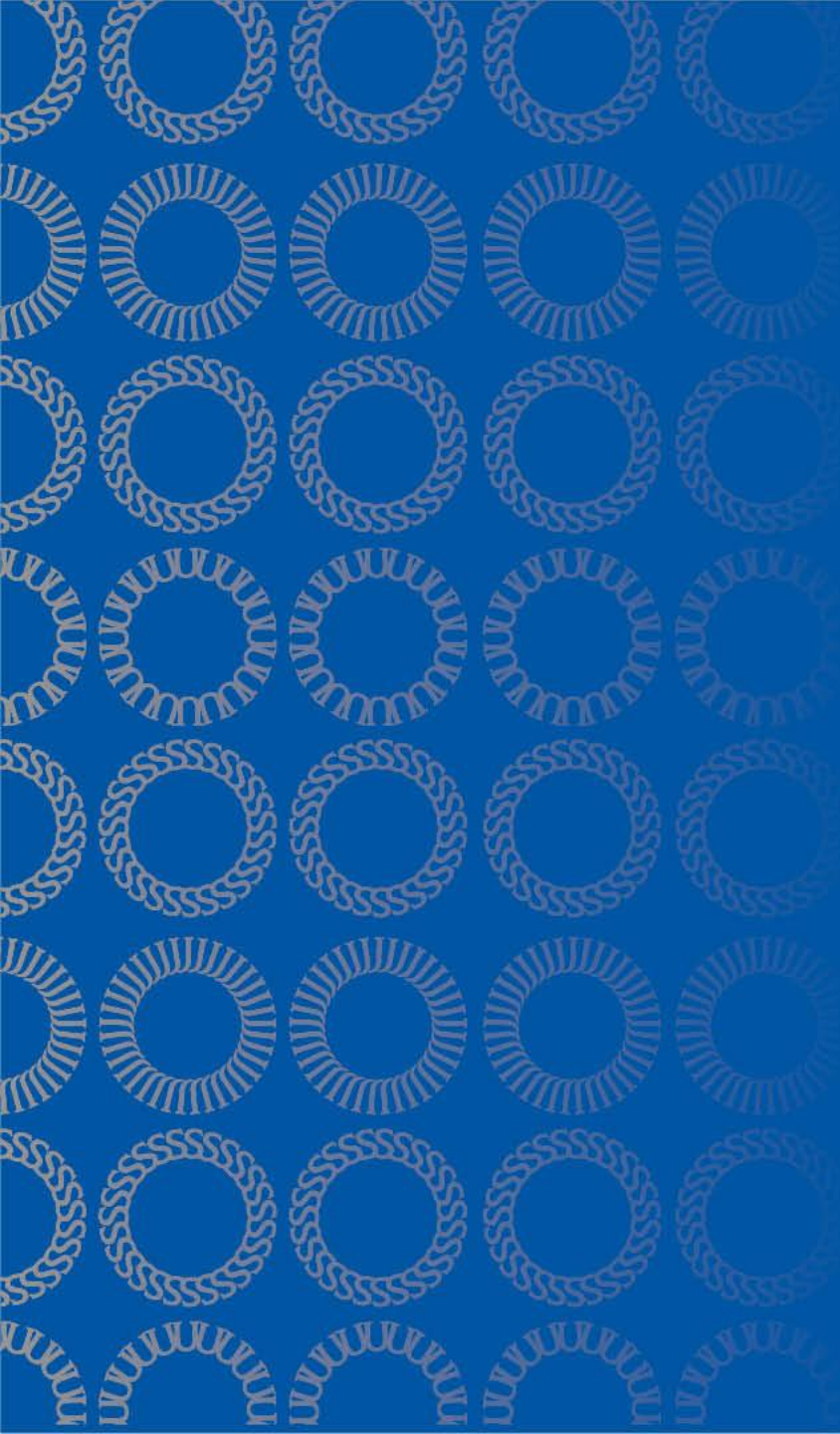




DATA 255

Deep Learning and Computer Vision in Applied Data Intelligence

Sept 9, 2026

Taehee Jeong, Ph.D.

San Jose State University

Today's Topics(1)

- Model optimization: Backward propagation

Model optimizing procedure

1. Initialize parameters
2. Run the optimization loop
 1. Forward propagation to compute the **loss** function
 2. Backward propagation to compute the **gradients** with respect to the loss function
 3. Using the gradients, **update** your parameter with the gradient descent update rule
3. Return the learned parameters

Source: deep learning, Andrew Ng

Forward pass for case2

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]}$$

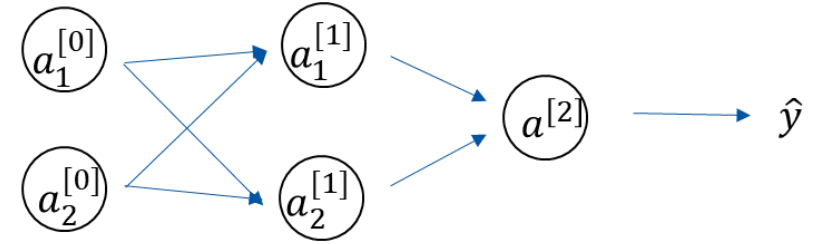
$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} \cdot a_1^{[1]} + W_2^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$



Input layer

Hidden layer

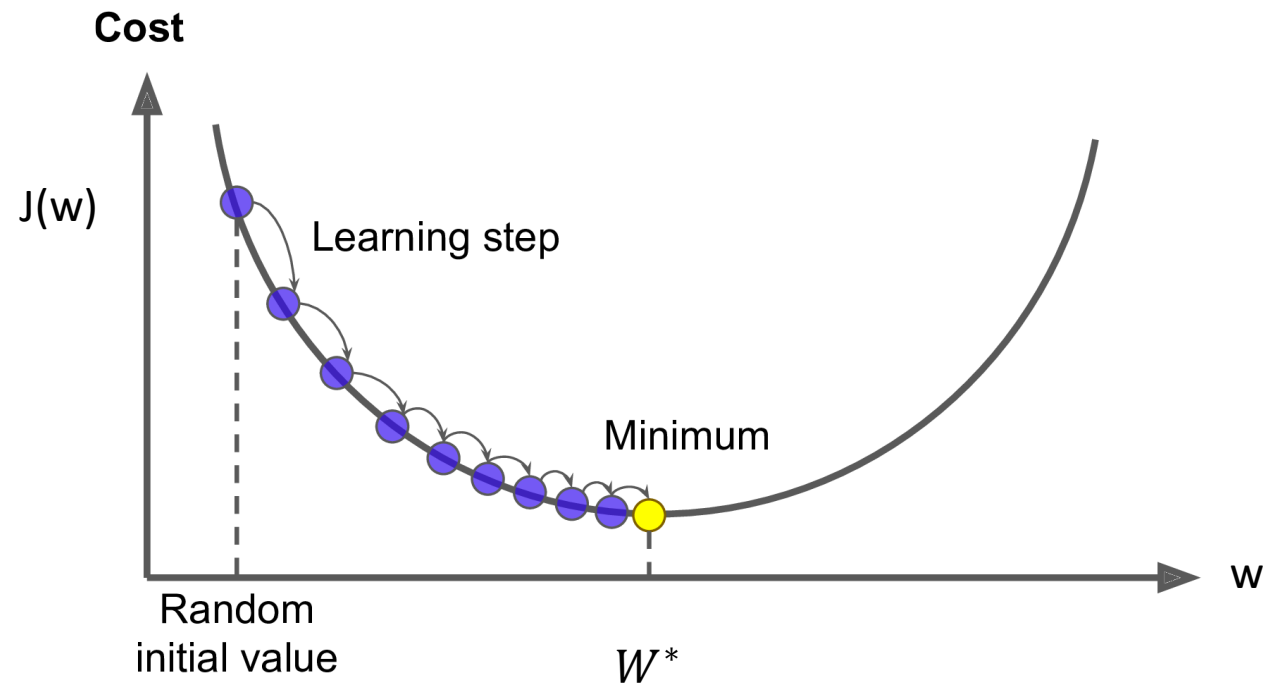
Output layer

Cost function(binary cross entropy)

Cost function J

$$J = -\frac{1}{m} \sum_{i=1}^m [y^i \log(\hat{y}^i) + (1 - y^i) \log(1 - \hat{y}^i)]$$

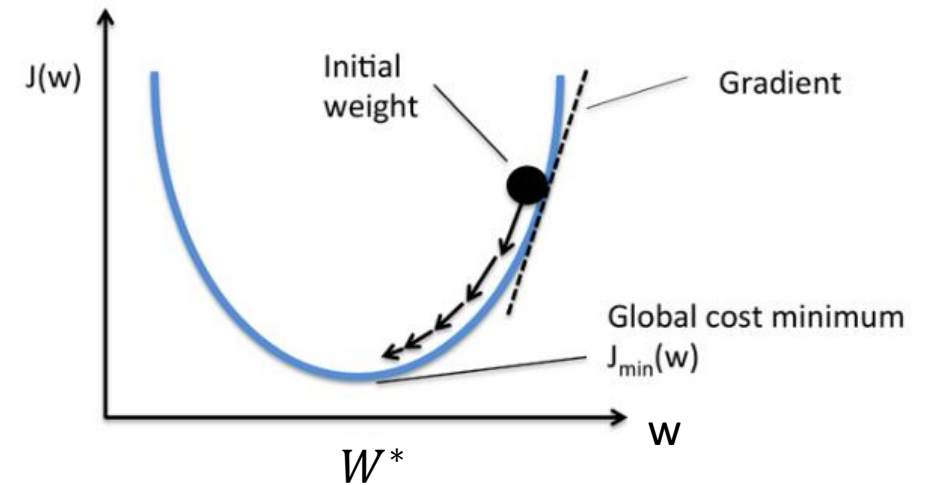
Gradient Descent



$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Gradient descent method

- To find the weights that minimize a cost function, gradient descent is commonly used as an optimization algorithm.
- Gradient(vector of partial derivatives) gives the input direction in which the function **decreases**
 - Step0: pick a random starting point
 - Step1: compute the gradient
 - Step2: take a small step in the direction of the gradient
 - Step3: repeat with the new starting point

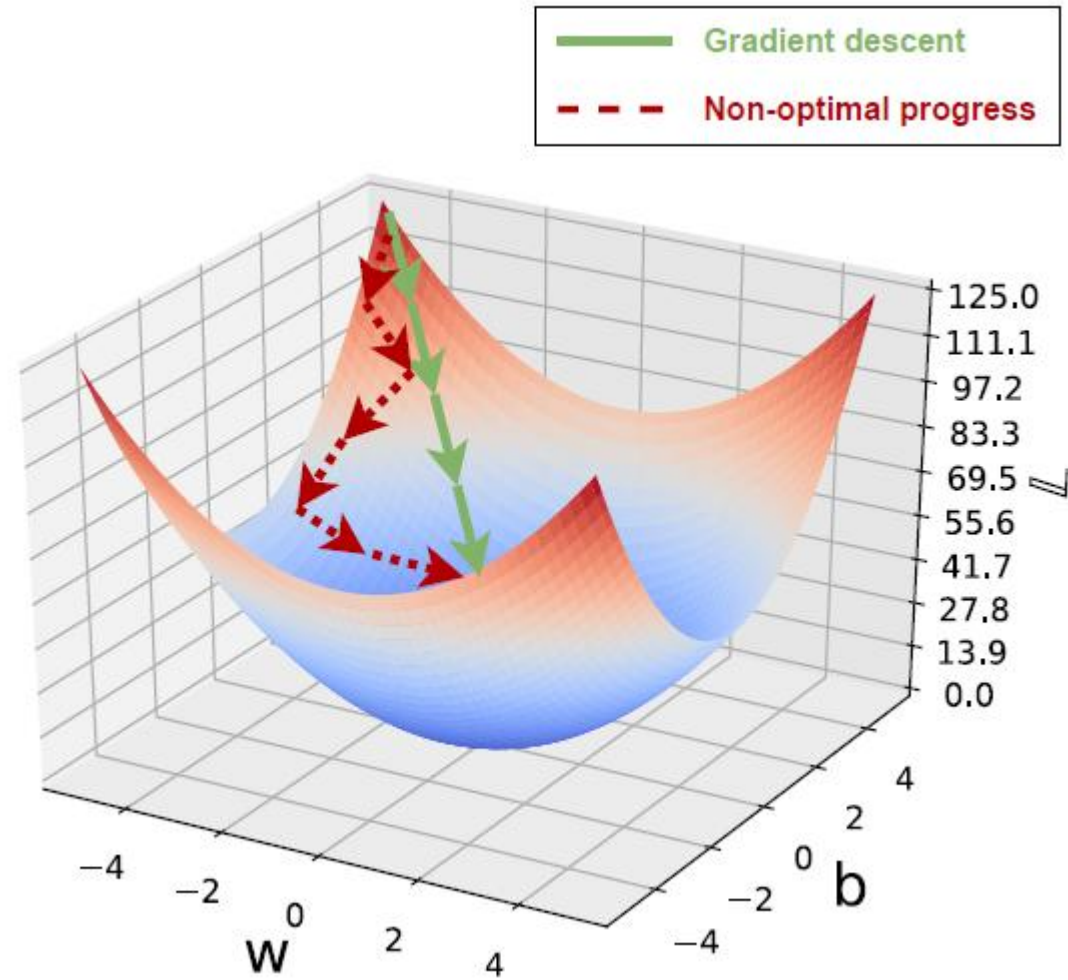


Gradient Descent

$$w = w - r \nabla_w L$$

$$\nabla_w L = \left[\frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k \right]$$

$$\partial w_{jk}^l := \partial w_{jk}^l - r \frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k$$



Source: Math and Architectures of Deep Learning, Krishnedu Chauhury

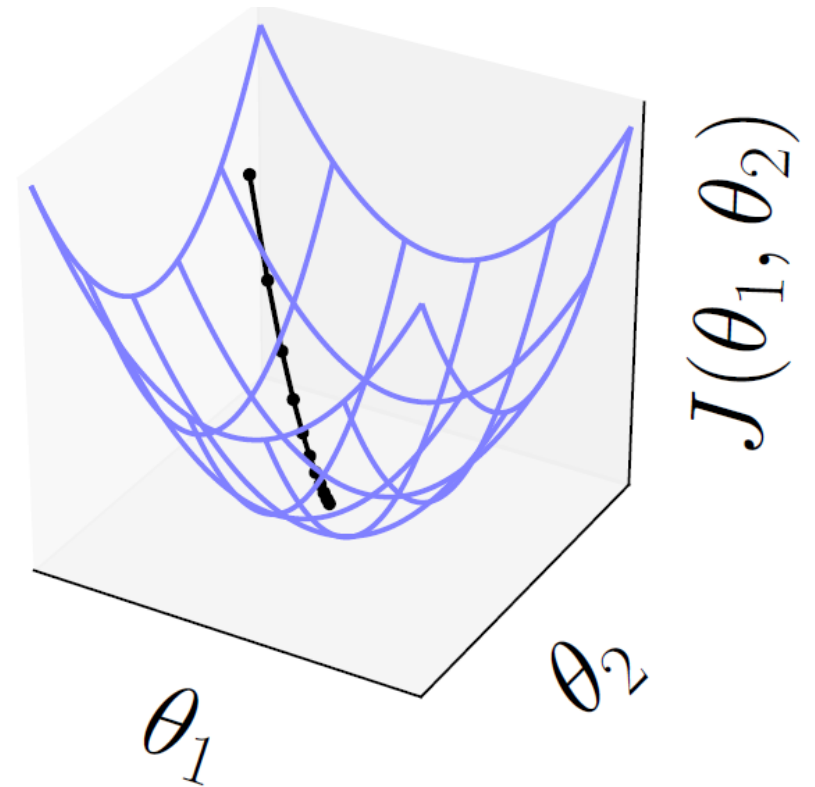
Gradient Descent

$$w = w - r \nabla_w L$$

$$\nabla_w L = \left[\frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k \right]$$

$$\partial w_{jk}^l := \partial w_{jk}^l - r \frac{\partial L}{\partial w_{jk}^l} \text{ for all } l, j, k$$

Source: Math and Architectures of Deep Learning, Krishnendu Chauhury



Source: "Adversarial Machine Learning," by Ian Goodfellow, ICLR Keynote, 2019

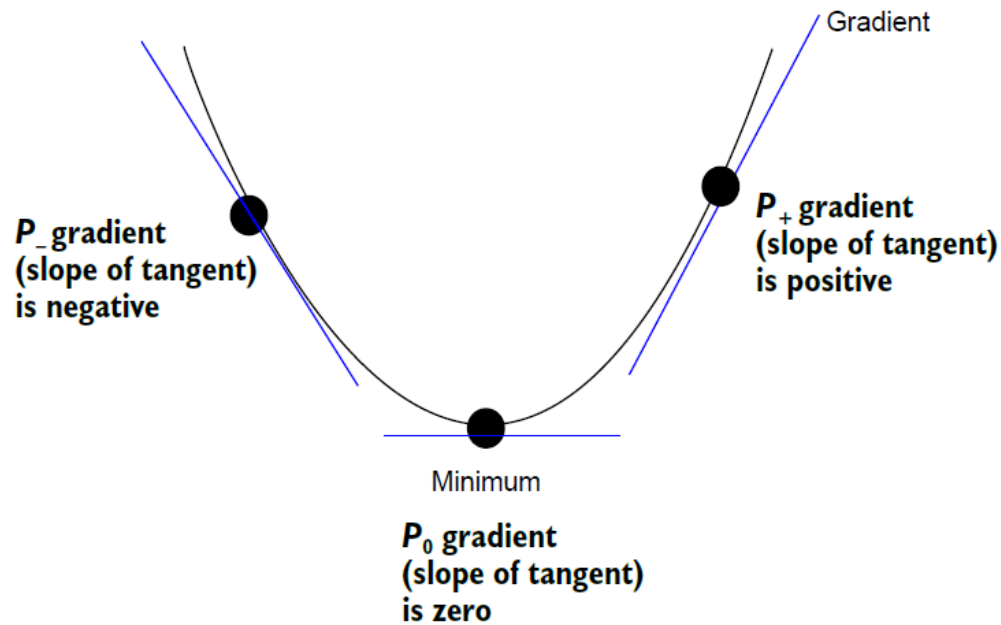
Gradient Descent

Find optimum W to minimize J

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

- W : weight or Model parameters
- J : cost function or loss function
- α : learning rate or step size
- $\frac{\partial J}{\partial W}$: gradient

Gradient is zero at the minimum



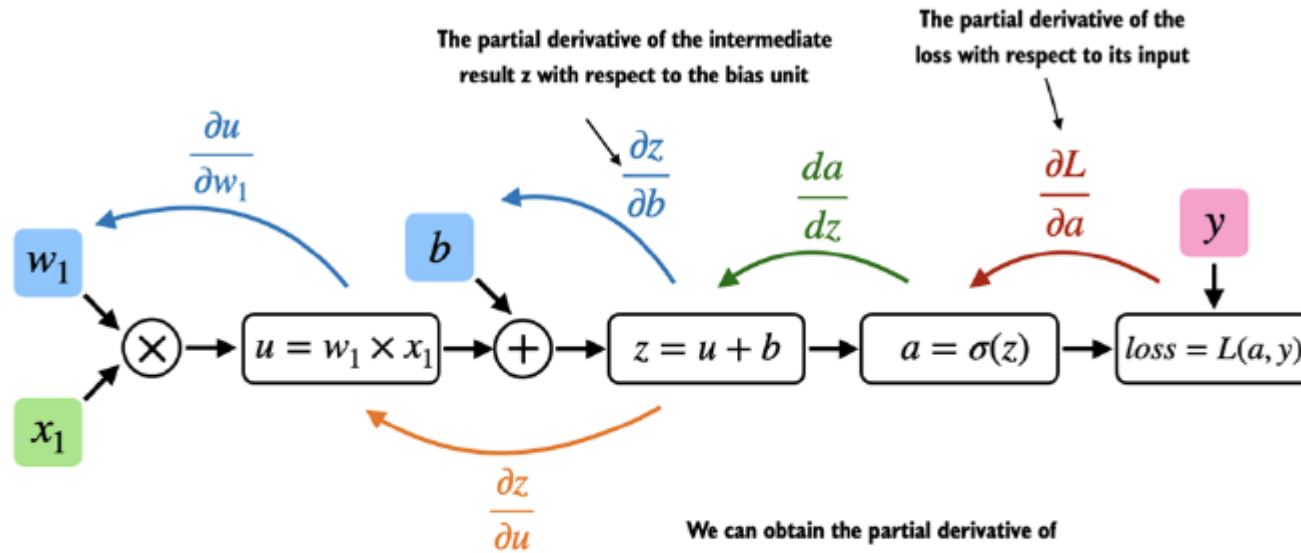
Any optimum (that is, maximum or minimum) of a function is a point of inflection. This means the function turns around at the optimum point.

Backward propagation

- The derivative of the loss in forward pass needs to be computed with respect to the weights in all layers in the backwards pass.
- The main goal of the backward propagation is to learn the gradient of the loss function with respect to the different weights by using the chain rule.
- These gradients are used to update the weights.
- Since these gradients are learned in the backward direction, starting from the output node, this learning process is referred to as the backward pass.

Backward propagation

Backward propagation is a way of computing gradients of expressions through recursive application of chain rule.



$$\frac{\partial L}{\partial w_1} = \frac{\partial u}{\partial w_1} \times \frac{\partial z}{\partial u} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

We can obtain the partial derivative of the loss with respect to the trainable weight by chaining the individual partial derivative in the graph

$$\frac{\partial L}{\partial b} = \frac{\partial z}{\partial b} \times \frac{da}{dz} \times \frac{\partial L}{\partial a}$$

Similar to above, we can compute the partial derivative of the trainable derivative by applying the chain rule

Derivative formula

Product rule

$$h(x) = f(x) \cdot g(x)$$

$$\frac{d}{dx} h(x) = \left[\frac{d}{dx} f(x) \right] \cdot g(x) + f(x) \cdot \left[\frac{d}{dx} g(x) \right]$$

$$h(x) = f(g(x))$$

$$\frac{d}{dx} [f(g(x))] = f'(g(x)) \cdot g'(x)$$

Chain rule

If $y = f(u)$, where $u = g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Derivative formula

Product rule

$$h(x) = f(x) \cdot g(x)$$

$$\frac{\partial h}{\partial x} = f' \cdot g + f \cdot g'$$

$$h(x) = \frac{f(x)}{g(x)}$$

$$h = f \cdot g^{-1}$$

$$\frac{\partial h}{\partial x} = f' \cdot g^{-1} + f \cdot \{-g^{-2}\} \cdot g'$$

$$= \frac{f'}{g} - \frac{f \cdot g'}{g^2}$$

$$= \frac{f' \cdot g - f \cdot g'}{g^2}$$

Sigmoid derivative via chain rule

Sigmoid function: $\sigma(x) = \frac{1}{1 + \exp(-x)}$

Chain rule: $\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$

$$\frac{d}{dx}\sigma(x) = \frac{d}{dx}\left[\frac{1}{1 + \exp(-x)}\right] = \frac{d}{dx}[(1 + \exp(-x))^{-1}]$$

$$\frac{d}{dx}\sigma(x) = -1 * (1 + \exp(-x))^{-2} * (-\exp(-x))$$

$$\frac{d}{dx}\sigma(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2}$$

$$\begin{aligned}\frac{d}{dx}e^x &= e^x \\ \frac{d}{dx}e^{-x} &= -e^{-x}\end{aligned}$$

Sigmoid derivative via chain rule

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$\frac{d}{dx} \sigma(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2}$$

$$\frac{d}{dx} \sigma(x) = \frac{1}{1 + \exp(-x)} * \frac{\exp(-x)}{1 + \exp(-x)}$$

$$\frac{d}{dx} \sigma(x) = \frac{1}{1 + \exp(-x)} * \frac{1 + \exp(-x) - 1}{1 + \exp(-x)}$$

$$\frac{d}{dx} \sigma(x) = \frac{1}{1 + \exp(-x)} * \left[1 - \frac{1}{1 + \exp(-x)} \right]$$

$$\frac{d}{dx} \sigma(x) = \sigma(x) * [1 - \sigma(x)]$$

Derivatives of Sigmoid function

$$g(z) = \frac{1}{1 + \exp(-z)}$$

$$\frac{d}{dx} g(x) = \frac{\exp(-x)}{(1 + \exp(-x))^2}$$

$$g'(z) = g(z)(1 - g(z))$$

$$\begin{aligned}\frac{d}{dx} e^x &= e^x \\ \frac{d}{dx} e^{-x} &= -e^{-x}\end{aligned}$$

Derivatives of Tanh function

$$g(z) = a = \tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

$$\frac{d}{dz} g(z) = \frac{\exp(z) + \exp(-z)}{\exp(z) + \exp(-z)} - \frac{\exp(z) - \exp(-z)}{(\exp(z) + \exp(-z))^2} \cdot (\exp(z) + \exp(-z))$$

$$\frac{d}{dz} g(z) = 1 - \frac{(\exp(z) - \exp(-z))^2}{(\exp(z) + \exp(-z))^2}$$

$$\frac{d}{dz} g(z) = 1 - (\tanh(z))^2$$

$$g'(z) = 1 - a^2$$

$$g'(z) = 1 - g(z)^2$$

$$\begin{aligned} \frac{d}{dx} e^x &= e^x \\ \frac{d}{dx} e^{-x} &= -e^{-x} \end{aligned}$$

Derivatives of Tanh function

$$g(z) = \tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}$$

$$g'(z) = 1 - g(z)^2$$

$$\frac{d}{dx} e^x = e^x$$
$$\frac{d}{dx} e^{-x} = -e^{-x}$$

* You should not use derivatives of trigonometric functions.

Derivatives of ReLU and Leaky ReLU

ReLU

$$a = g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

Leaky ReLU

$$a = g(z) = \max(0.01z, z)$$

$$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

Derivatives of ELU

$$a = ELU(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

$$g'(z) = \begin{cases} \alpha \exp(z) & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

Derivative of binary cross entropy

$$L(a, y) = -[y \log a + (1 - y) \log(1 - a)]$$

$$\frac{\partial L}{\partial W_0} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial Z} \frac{\partial Z}{\partial W_0}$$

$$\frac{\partial L}{\partial W_0} = \left[-\frac{y}{a} + \frac{1 - y}{(1 - a)} \right] [a * (1 - a)] \cdot 1$$

$$\frac{\partial L}{\partial W_0} = [-y(1 - a) + a(1 - y)] \cdot 1$$

$$\frac{\partial L}{\partial W_0} = (a - y) \cdot 1$$

$$\frac{d}{dx} \log(x) = \frac{1}{x}$$

$$\frac{d}{dx} \log(1 - x) = \frac{-1}{1 - x}$$

$$a = \sigma(z)$$

$$\frac{d}{dz} \sigma(z) = \sigma(z) * [1 - \sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Derivative of binary cross entropy

$$L(a, y) = -[y \log a + (1 - y) \log(1 - a)]$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial Z} \frac{\partial Z}{\partial W_1}$$

$$\frac{\partial L}{\partial W_1} = \left[-\frac{y}{a} + \frac{1-y}{(1-a)} \right] [a * (1-a)] \cdot x_1$$

$$\frac{\partial J}{\partial W_1} = [-y(1-a) + a(1-y)] \cdot x_1$$

$$\frac{\partial J}{\partial W_1} = (a - y) \cdot x_1$$

$$\frac{d}{dx} \log(x) = \frac{1}{x}$$

$$\frac{d}{dx} \log(1-x) = \frac{-1}{1-x}$$

$$a = \sigma(z)$$

$$\frac{d}{dz} \sigma(z) = \sigma(z) * [1 - \sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Derivative of binary cross entropy

$$L(a, y) = -[y \log a + (1 - y) \log(1 - a)]$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial Z} \frac{\partial Z}{\partial W_2}$$

$$\frac{\partial L}{\partial W_2} = \left[-\frac{y}{a} + \frac{1 - y}{(1 - a)} \right] [a * (1 - \hat{y})] \cdot x_2$$

$$\frac{\partial J}{\partial W_2} = [-y(1 - a) + a(1 - y)] \cdot x_2$$

$$\frac{\partial J}{\partial W_2} = (a - y) \cdot x_2$$

$$\frac{d}{dx} \log(x) = \frac{1}{x}$$

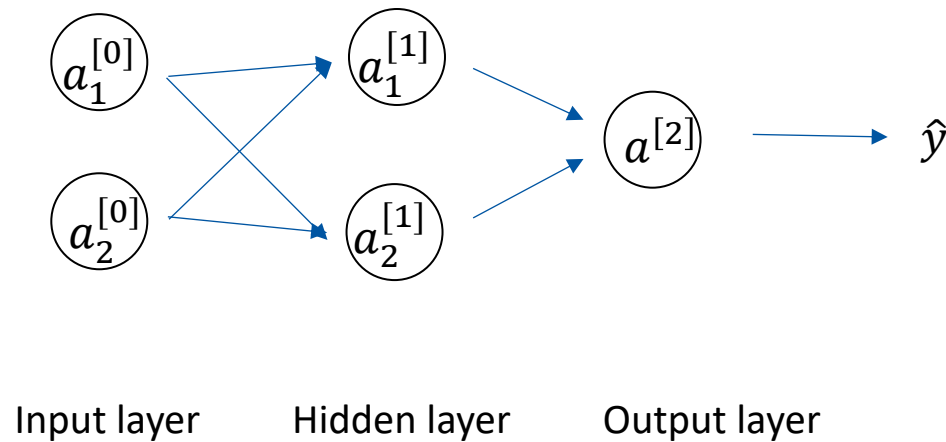
$$\frac{d}{dx} \log(1 - x) = \frac{-1}{1 - x}$$

$$a = \sigma(z)$$

$$\frac{d}{dz} \sigma(z) = \sigma(z) * [1 - \sigma(z)]$$

$$Z = W_0 + W_1 x_1 + W_2 x_2$$

Forward pass for case2

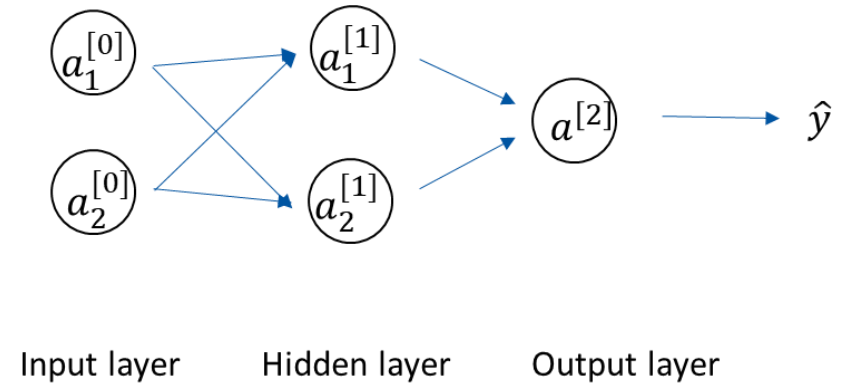
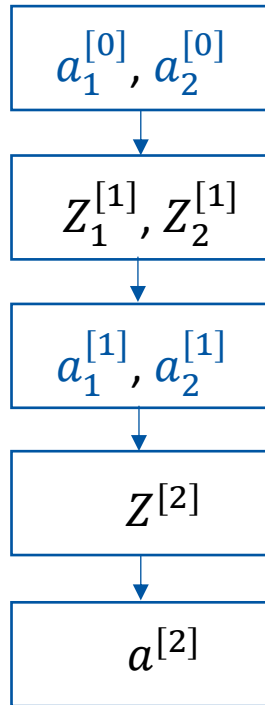


Parameters: $W^{[1]}, W^{[2]}$

Cost function: $L(W^{[1]}, W^{[2]})$

$$J = \frac{1}{m} \sum_{i=1}^m L(a^{[2]}, y)$$

Forward propagation



Forward propagation

1st layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$$

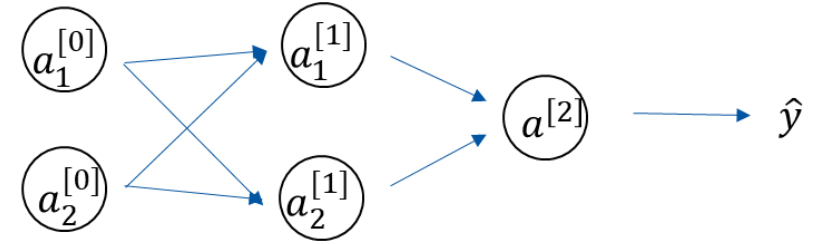
$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

2nd layer

$$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$



Input layer

Hidden layer

Output layer

Derivative of $Z^{[2]}$

backpropagation for binary entropy loss

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$dZ^{[2]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}}$$

$$dZ^{[2]} = (a^{[2]} - y)$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_0^{[2]}$

backpropagation for binary entropy loss

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(a, y)}{\partial W_0^{[2]}} = dW_0^{[2]}$$

$$\begin{aligned} dW_0^{[2]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}} \\ &= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_0^{[2]}} \end{aligned}$$

$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_1^{[2]}$

backpropagation for binary classification

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(a^{[2]}, y)}{\partial W_1^{[2]}} = dW_1^{[2]}$$

$$\begin{aligned} dW_1^{[2]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}} \\ &= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_1^{[2]}} \end{aligned}$$

$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_2^{[2]}$

backpropagation for binary classification

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$\frac{\partial L(a^{[2]}, y)}{\partial W_2^{[2]}} = dW_2^{[2]}$$

$$\begin{aligned} dW_2^{[2]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}} \\ &= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial W_2^{[2]}} \end{aligned}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

**Element wise multiplication*

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of Parameters in 2nd layer

backpropagation for binary entropy loss

$$L(a^{[2]}, y) = -[y * \log(a^{[2]}) + (1 - y) * \log(1 - a^{[2]})]$$

$$dZ^{[2]} = a^{[2]} - y$$

$$dW_0^{[2]} = dZ^{[2]} \cdot 1$$

$$dW_1^{[2]} = dZ^{[2]} \cdot a_1^{[1]}$$

$$dW_2^{[2]} = dZ^{[2]} \cdot a_2^{[1]}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $da_1^{[1]}$

$$\frac{\partial L(\hat{y}, y)}{\partial a_1^{[1]}} = da_1^{[1]}$$

$$da_1^{[1]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_1^{[1]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial a_1^{[1]}}$$

$$da_1^{[1]} = dZ^{[2]} \cdot W_1^{[2]}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $da_2^{[1]}$

$$\frac{\partial L(\hat{y}, y)}{\partial a_1^{[1]}} = da_1^{[1]}$$

$$da_2^{[1]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_2^{[1]}}$$

$$= dZ^{[2]} \frac{\partial Z^{[2]}}{\partial a_2^{[1]}}$$

$$da_2^{[1]} = dZ^{[2]} \cdot W_2^{[2]}$$

$$da_2^{[1]} = \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_2^{[1]}}$$

Derivative of $dZ_1^{[1]}$

$$\begin{aligned}\frac{\partial L(\hat{y}, y)}{\partial Z_1^{[1]}} &= dZ_1^{[1]} \\ dZ_1^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_1^{[1]}} \frac{\partial a_1^{[1]}}{\partial Z_1^{[1]}} \\ &= da_1^{[1]} \frac{\partial a_1^{[1]}}{\partial Z_1^{[1]}} \\ dZ_1^{[1]} &= da_1^{[1]} * a_1^{[1]} * [1 - a_1^{[1]}]\end{aligned}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $dZ_2^{[1]}$

$$\begin{aligned}\frac{\partial L(\hat{y}, y)}{\partial Z_2^{[1]}} &= dZ_2^{[1]} \\ dZ_2^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_2^{[1]}} \frac{\partial a_2^{[1]}}{\partial Z_2^{[1]}} \\ &= da_2^{[1]} \frac{\partial a_2^{[1]}}{\partial Z_2^{[1]}} \\ dZ_2^{[1]} &= da_2^{[1]} * a_2^{[1]} * [1 - a_2^{[1]}]\end{aligned}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_{10}^{[1]}$

$$\frac{\partial L(a, y)}{\partial W_{10}^{[1]}} = dW_{10}^{[1]}$$

$$\begin{aligned} dW_{10}^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_1^{[1]}} \frac{\partial a_1^{[1]}}{\partial Z_1^{[1]}} \frac{\partial Z_1^{[1]}}{\partial W_{10}^{[1]}} \\ &= dZ_1^{[1]} \frac{\partial Z_1^{[1]}}{\partial W_{10}^{[1]}} \end{aligned}$$

$$dW_{10}^{[1]} = dZ_1^{[1]} \cdot 1$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_{11}^{[1]}$

$$\frac{\partial L(a, y)}{\partial W_{11}^{[1]}} = dW_{11}^{[1]}$$

$$\begin{aligned} dW_{11}^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_1^{[1]}} \frac{\partial a_1^{[1]}}{\partial Z_1^{[1]}} \frac{\partial Z_1^{[1]}}{\partial W_{11}^{[1]}} \\ &= dZ_1^{[1]} \frac{\partial Z_1^{[1]}}{\partial W_{11}^{[1]}} \end{aligned}$$

$$dW_{11}^{[1]} = dZ_1^{[1]} \cdot a_1^{[0]}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_{12}^{[1]}$

$$\frac{\partial L(a, y)}{\partial W_{12}^{[1]}} = dW_{12}^{[1]}$$

$$\begin{aligned} dW_{12}^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_1^{[1]}} \frac{\partial a_1^{[1]}}{\partial Z_1^{[1]}} \frac{\partial Z_1^{[1]}}{\partial W_{12}^{[1]}} \\ &= dZ_1^{[1]} \frac{\partial Z_1^{[1]}}{\partial W_{12}^{[1]}} \end{aligned}$$

$$dW_{12}^{[1]} = dZ_1^{[1]} \cdot a_2^{[0]}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_{20}^{[1]}$

$$\frac{\partial L(a, y)}{\partial W_{20}^{[1]}} = dW_{20}^{[1]}$$

$$\begin{aligned} dW_{20}^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_2^{[1]}} \frac{\partial a_2^{[1]}}{\partial Z_2^{[1]}} \frac{\partial Z_2^{[1]}}{\partial W_{20}^{[1]}} \\ &= dZ_2^{[1]} \frac{\partial Z_2^{[1]}}{\partial W_{20}^{[1]}} \end{aligned}$$

$$dW_{20}^{[1]} = dZ_2^{[1]} \cdot 1$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_{21}^{[1]}$

$$\frac{\partial L(a, y)}{\partial W_{21}^{[1]}} = dW_{21}^{[1]}$$

$$\begin{aligned} dW_{21}^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_2^{[1]}} \frac{\partial a_2^{[1]}}{\partial Z_2^{[1]}} \frac{\partial Z_2^{[1]}}{\partial W_{21}^{[1]}} \\ &= dZ_2^{[1]} \frac{\partial Z_2^{[1]}}{\partial W_{21}^{[1]}} \end{aligned}$$

$$dW_{21}^{[1]} = dZ_2^{[1]} \cdot a_1^{[0]}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Derivative of $W_{22}^{[1]}$

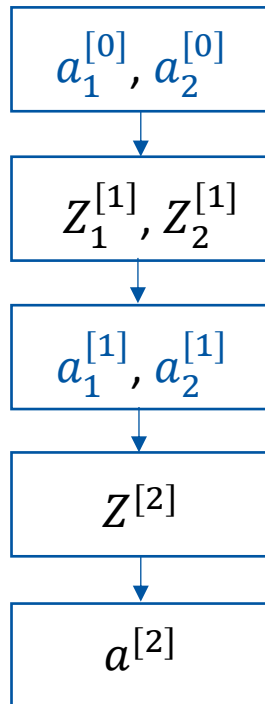
$$\frac{\partial L(a, y)}{\partial W_{22}^{[1]}} = dW_{22}^{[1]}$$

$$\begin{aligned} dW_{22}^{[1]} &= \frac{\partial L}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial a_2^{[1]}} \frac{\partial a_2^{[1]}}{\partial Z_2^{[1]}} \frac{\partial Z_2^{[1]}}{\partial W_{22}^{[1]}} \\ &= dZ_2^{[1]} \frac{\partial Z_2^{[1]}}{\partial W_{22}^{[1]}} \end{aligned}$$

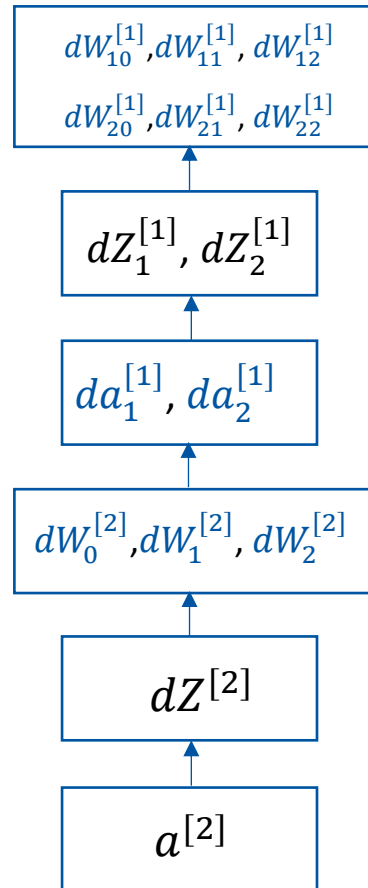
$$dW_{22}^{[1]} = dZ_2^{[1]} \cdot a_2^{[0]}$$

1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$
	$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$
	$a_1^{[1]} = \sigma(Z_1^{[1]})$
	$a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$
	$a^{[2]} = \sigma(Z^{[2]})$

Forward propagation



Backward propagation



1 st layer	$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} a_1^{[0]} + W_{12}^{[1]} a_2^{[0]}$ $Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} a_1^{[0]} + W_{22}^{[1]} a_2^{[0]}$ $a_1^{[1]} = \sigma(Z_1^{[1]})$ $a_2^{[1]} = \sigma(Z_2^{[1]})$
2 nd layer	$Z^{[2]} = W_0^{[2]} + W_1^{[2]} a_1^{[1]} + W_2^{[2]} a_2^{[1]}$ $a^{[2]} = \sigma(Z^{[2]})$

Cache forward pass variables

To compute the backward pass,

it is very helpful to have some of the variables that were used in the backward pass or forward pass.

In practice you want to structure your code so that you cache these variables, and so that they are available during backpropagation.

Autograd: PyTorch automatic gradient computation

- For a specific model architecture, we computed the gradient using calculus.
- This approach does not scale to more complex models with millions of weights and perhaps nonlinear complex functions.
- For scalability, we use an automatic differentiation software library like PyTorch Autograd.
- Users of the library need not worry about how to compute the gradients—they just construct the model function.
- Once the function is specified, PyTorch figures out how to compute its gradient through the Autograd technology.

Differential equation (Derivative)

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

indicate the rate of change of a function with respect to that variable surrounding an infinitesimally small region near a particular point

$$f(x+h) = f(x) + h \frac{df(x)}{dx}$$

*when h is very small,
then the function is well approximated by a straight line,
and the derivative is its slope*